PART B - SYSTEM ADMINISTRATOR DOCUMENTATION

Student Housing Platform

Administrator Manual

Purpose
A concise operational guide to install, configure, administer, and maintain the Alfa student accommodation platform.

System profile
Next.js 14 application | Supabase Auth and PostgreSQL | Role-based housing workflows

Alpha / demonstration release | 26 October 2026

01 / SYSTEM OVERVIEW

What the system does

The platform supports student accommodation discovery and operations. Students browse listings, request bookings or leases, and use housing tools. Owners and agents manage accommodation workflows; universities support student oversight and communications. Admins oversee platform operations.

People
Student · Owner · Agent
University · Admin

›

Next.js 14
Pages · role dashboards
API routes

›

Data services
Supabase Auth / Postgres
Demo JSON fallback

Main components

- **Web application:** Next.js, React and Tailwind CSS; local development runs on port 3000 by default.
- **API:** Next.js route handlers for listings, profiles, bookings, messages, lease requests and student-hub workflows.
- **Identity and persistence:** Supabase Auth and PostgreSQL, accessed through client and server-side helpers.
- **Optional integrations:** Google Maps/Places keys for address and map features. Some modules include demo/local JSON data.

Roles and areas

Role	Typical use
Student	Search, bookings, messaging, student hub
Owner / Agent	Listings, enquiries, leases, communications
University	Student oversight and accommodation support
Admin	Operational oversight and role-specific access

Administrator responsibilities

Access: provision users and confirm roles; restrict privileged credentials.

Data: monitor Supabase tables, schema changes, and application errors.

Continuity: maintain backups, dependencies, environment secrets, and deployment settings.

Release status: This is an alpha/demo prototype. Validate permissions, data protection, backups, and operational controls before using real personal data or deploying publicly.

02 / REQUIREMENTS AND SETUP

Prepare the environment

Requirements

- Windows, macOS, or Linux development machine; 2 CPU cores and 4 GB RAM recommended for local development.
- Node.js 18.17 or later (Next.js 14 runtime baseline) and npm.
- Git and a modern browser; internet access for package installation and hosted Supabase.
- Supabase project for authentication and PostgreSQL-backed features. Google Maps/Places API key is optional.

Install

1. Obtain the repository and open a terminal at its root.
2. Install packages: `npm install`.
3. Create `.env.local` at the project root; fill in the values below.
4. In Supabase SQL Editor, apply `sql/complete_app_schema.sql` to create the documented tables/indexes.
5. Start: `npm run dev`; browse to `http://localhost:3000`.

Environment settings

```
NEXT_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<publishable-key>
SUPABASE_SERVICE_ROLE=<server-only-service-role-secret>

# Optional: enable map/address autocomplete
NEXT_PUBLIC_GOOGLE_PLACE_API_KEY=<restricted-browser-key>
# Alternative map variable accepted by the app:
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=<restricted-browser-key>
```

Secret handling

Never commit `.env.local`. The `NEXT_PUBLIC_` values are exposed to the browser; use only the Supabase publishable/anon key and a domain-restricted Maps key. `SUPABASE_SERVICE_ROLE` bypasses normal database protections: keep it server-side, use it only for trusted admin scripts, and rotate it if exposed.

Database and storage

Run the provided schema in Supabase before testing live workflows. Configure database access policies and any required storage bucket/policies for the actual deployment. Apply additional SQL migrations in `sql/` when the corresponding feature or schema change requires them.

Check startup: If Supabase is not configured, some demo pages may still render, but sign-in and database-backed functions will not work reliably. Treat fallback JSON as demo behavior, not a production database.

03 / DAY-TO-DAY ADMINISTRATION

Accounts, roles, and records

Manage accounts

1. Create or invite users through Supabase Auth using the project's approved authentication process.
2. Set the intended role in user metadata and keep the associated `profiles` row aligned (role values: `student`, `agent`, `owner`, `university`, `admin`).
3. Verify that the user can sign in and sees the correct role workspace. Remove access promptly when no longer required.

Role normalization and application permission checks are in `lib/permissions.js`. Verify database row-level security independently; UI/API role handling alone is not a security boundary.

Demo provisioning scripts

For isolated demonstrations only, scripts in `scripts/` can create sample student, owner, and admin identities using the service-role secret. Run from the repository root, for example:

```
node scripts/create-demo-student.js
node scripts/create-demo-owner.js
node scripts/create-demo-admin.js
```

Some scripts use fixed demo identities/passwords; do not use these accounts or scripts in production. Change demo credentials immediately, and do not share generated credentials.

Data operations

Task	Recommended procedure
Inspect live data	Use Supabase Dashboard (Table Editor/SQL Editor) with an appropriately privileged account. Confirm project and environment before editing.
Schema updates	Review the relevant SQL file in `sql/`, take a backup, apply in a controlled environment, then verify tables and a representative workflow.
Backups / restore	Enable Supabase backups appropriate to the chosen plan; periodically export critical tables or database snapshots to access-controlled storage. Test restore procedures before relying on them.
Demo fallback files	Sample/local JSON is under `data/`. Back it up only if those local demo records are needed; it is not a substitute for a managed live database backup.

Routine maintenance checklist

Review Supabase Auth users and role assignments; remove stale demo users. Check database/storage usage and error logs.

Keep dependencies patched; test changes with `npm run build` before release. Rotate secrets after staff changes or suspected exposure.

Data protection: Store only necessary personal information. Restrict dashboard and service-role access, use least privilege, and follow the organisation's retention and privacy policies.

04 / TROUBLESHOOTING

Common problems and practical fixes

Symptom	Likely cause	Action
Dependencies fail to install or app will not start	Unsupported Node version, incomplete install, or occupied port	Check `node -v` (18.17+), run `npm install` from the project root, then retry. If port 3000 is occupied, use the port reported by Next.js.
Sign-in unavailable / Supabase warning in terminal	Missing or incorrect URL/key	Check `.env.local` variable names and values; restart the dev server after edits. Use the publishable/anon key for the client; never expose the service-role secret.
Database requests fail, table missing, or empty live views	Schema not applied, wrong Supabase project, or access policy blocks request	Confirm the configured project URL; apply `sql/complete_app_schema.sql` and relevant migrations. Review Supabase logs, table definitions, and RLS policies.
User lands on wrong dashboard or receives access denied	Role metadata/profile mismatch, expired session, or permission rule	Confirm Auth user metadata and matching `profiles.role`; sign out/in. Check allowed role and action in `lib/permissions.js`, then verify database policies.
Maps/address search do not load	Optional Maps key missing, restricted incorrectly, or API not enabled	Check one of the documented Google key variables; enable the required Maps/Places APIs and allow the app origin in the Google Cloud key restrictions.
Changes appear to vanish in demo/local mode	Feature may read from static JSON/local fallback rather than Supabase	Confirm Supabase configuration and feature's persistence implementation. Do not assume local fallback data is shared or durable.
Production build fails	Compile/type/lint error or environment-dependent configuration	Run `npm run build` and inspect the first reported error. Correct the source/configuration and retest; do not suppress errors to force deployment.

Escalation checklist

- Record timestamp, environment, affected route, and reproduction steps.
- Review terminal/hosting logs and Supabase Auth, API, and database logs.
- Check recent schema, dependency, and environment changes.
- Do not include passwords, tokens, or service-role keys in logs or support tickets.

Operational references

- `README.md` - quick start and project overview
- `sql/complete_app_schema.sql` - application database schema
- `sql/` - feature-specific migrations and setup notes
- `lib/permissions.js` - application role and permission rules
- `scripts/` - demo data/account utilities

Production readiness: Before public launch, independently test row-level security, authentication, file storage access, privacy controls, backup restoration, monitoring, and incident response. The alpha project documentation does not certify these controls.